

Lab 06

Licenciatura em Engenharia Informática e Multimédia

Sensores e Atuadores - Ano Letivo 2022/2023

Turma 11D / Grupo 04:

Nomes

Números

Fábio Pestana

50756

Rodrigo Nicolau

50728

Robert Sbircea

50732

Introdução

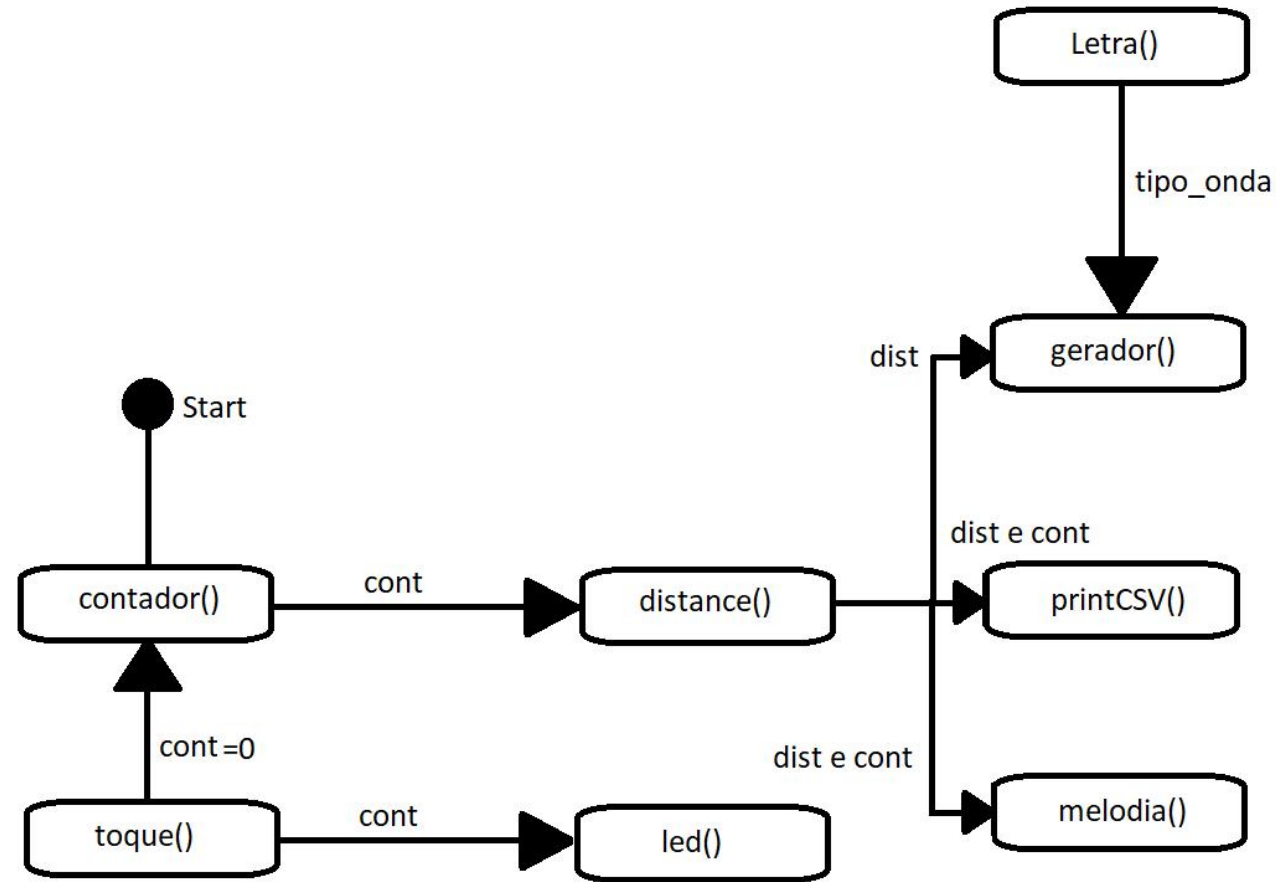
Introduzir aos alunos conceitos práticos sobre a programação com máquinas de estados e trabalhar com diagramas UML.

O trabalho consiste em criar 7 máquinas de estados e inseri-las num sistema multi-tarefa.

O contexto escolhido pelo grupo foi:

Quando o botão S1 é clicado, é incrementado um contador que permite ligar o sonar. Quando o sonar deteta um objeto a mais de 50 cm, no plotter é possível visualizar um dos três tipos de ondas (sinusoidal, triangular ou quadrada), quando a distância é menor que 150 cm, na consola aparece, em formato CSV, a distancia (medida pelo sonar), o valor do contador e o piezo começa a emitir som. O sonar só desliga quando o botão S2 é pressionado, sendo que nesse instante um led acende (sempre que o contador é zero).

Introdução - Autómato



Código Lab 6

```

#define S1 2
#define S2 3
#define TRIGG 7
#define ECHO 8
#define PIEZO 4
#define n1 65
#define n2 104
#define n3 294
#define LED 9

//variavel global
int cont=0;
float dist;
int x=150;
char tipo_onda;

////////////////////
void setup() {
  Serial.begin(9600);
  pinMode(S1, INPUT_PULLUP);
  pinMode(S2, INPUT_PULLUP);
  pinMode (TRIGG, OUTPUT);
  pinMode (ECHO, INPUT);
  pinMode(PIEZO,OUTPUT);
  pinMode(LED,OUTPUT);
}

```

```

void contador(int pin)
{
  const int COUNTER = 0 , BETWEEN=1;
  static int state = COUNTER;
  static bool botao;
  switch (state)
  {
    case COUNTER:
      botao = digitalRead(pin);
      if(botao==LOW)
      {
        cont= cont + 1;
        state= BETWEEN;
      }
      break;
    case BETWEEN:
      botao = digitalRead(pin);
      if (botao==HIGH)
      {
        state=COUNTER;
      }
      break;
  }
}

```

```

void distance(int pinTrig, int pinEcho, int rate)
{
  const int WAIT=0, SEND_PULSE = 1 , READ_PULSE = 2 , DISTANCE = 3;
  static int state = SEND_PULSE;
  static unsigned long t0 = millis(), T = micros(), th=micros();
  rate=1000/rate;
  switch (state)
  {
    case WAIT:
      if ( millis()-t0>rate && cont >= 1)
      {
        t0=millis();
        T = micros();
        state = SEND_PULSE;
      }
      break;
    case SEND_PULSE:
      digitalWrite(pinTrig, HIGH);
      if ( micros()-T>10)
      {
        digitalWrite(pinTrig, LOW);
        state = READ_PULSE;
      }
      break;
    case READ_PULSE:
      if(digitalRead(pinEcho)==HIGH)
      {
        th=micros();
        state = DISTANCE;
      }
      break;
    case DISTANCE:
      if(digitalRead(pinEcho)==LOW)
      {
        dist = (micros()-th)/58.0;
        //Serial.println (dist); //feito apenas para testar o sonar
        t0=millis();
        state = WAIT;
      }
      break;
  }
}

```

```

void melodia(int duracao, int nota1, int nota2, int nota3){
  unsigned long T=duracao; //Tempo que toca cada nota
  static const int NOTA1=0, NOTA2=1, NOTA3=2, STOP=3;
  static int state=NOTA1;
  static unsigned long t0=micros();
  switch (state)
  {
    case NOTA1:
      if (millis()-t0>T && (dist<x) && (cont>=1)){
        tone(PIEZO, nota1);
        t0 = millis();
        state = NOTA2;
      }
      break;
    case NOTA2:
      if (millis()-t0>T){
        tone(PIEZO, nota2);
        t0 = millis();
        state = NOTA3;
      }
      break;
    case NOTA3:
      if (millis()-t0>T){
        tone(PIEZO, nota3);
        t0 = millis();
        state = STOP;
      }
    case STOP:
      if (millis()-t0>T){
        noTone(PIEZO);
        t0 = millis();
        state = NOTA1;
      }
      break;
  }
}

```

```

void letra()
{
    static const int IDLE = 0, READ = 1;
    static int state = IDLE;
    static char aux;
    switch (state)
    {
        case IDLE:
            if(Serial.available() > 0){
                aux = Serial.read();
                state = READ;
            }
            break;
        case READ:
            if(aux == 's' || aux == 'q' || aux == 't')
            {
                tipo_onda = aux;
                state = IDLE;
            }
            break;
    }
}

```

```

void gerador(int rate, int periodo, char tipoOnda){
    static const int IDLE=0, DECISAO=1, SINAL=2, QUAD=3, TRI=4;
    static unsigned long t=millis();
    static int state = IDLE;
    int T=periodo;
    rate = 1000/rate;
    //sinusoidal:
    float ydc=2.5, yac=2.5; //max 5 e min 0
    float onda=0;
    int Ts=1000*T/2; //semiperiod in milisegundos (quadrada e triangular)
    float ymin=0.0, ymax=5.0; //quadrada e triangular
    unsigned long tmin, tmax;

    float ystart, yend; //y value at start/end of semicycle
    float mapyscale=100;

    switch (state)
    {
        case IDLE:
            if(millis()-t > rate && dist >= x)
            {
                t=millis();
                state=DECISAO;
            }
            break;
        case DECISAO:
            if(tipo_onda == 's')
            {
                state=SINAL;
            }
            if(tipo_onda == 'q')
            {
                state=QUAD;
            }
            if(tipo_onda == 't')
            {
                state=TRI;
            }
            break;
    }
}

```

```

        case SINAL:
            t=millis();
            onda = ydc+yac*sin(6.28*t/1000/T);
            Serial.println(onda);
            if((tipo_onda != 's') || (dist >= x))
            {
                state=IDLE;
            }
            break;
        case QUAD: //ymin=0.0, ymax=5.0;; Ts=1000*T/2;
            t=millis();
            if((t/Ts)%2==0) Serial.println(ymin);
            else Serial.println(ymax);
            if((tipo_onda != 'q') || (dist >= x))
            {
                state=IDLE;
            }
            break;
        case TRI: ///ymin=0.0, ymax=5.0;; Ts=1000*T/2;
            tmin=t/Ts*Ts; //start time of each semicycle
            tmax=tmin+Ts; //end time of each semicycle
            if((tmin/Ts)%2==0)
            {
                ystart=ymin;
                yend=ymax;
            }
            else{
                ystart=ymax;
                yend=ymin;
            }
            t=millis();
            onda = map(t, tmin, tmax, mapyscale*ystart, mapyscale*yend)/mapyscale;
            Serial.println(onda);
            if((tipo_onda != 't') || (dist >= x))
            {
                state=IDLE;
            }
            break;
    }
}

```



```

void printCSV(int rate, float val1, int val2)
{
    static const int ESPERA=0, PRINT=1;
    static int state=ESPERA;
    static unsigned long t0= millis();
    rate=1000/rate;
    switch(state)
    {
        case ESPERA:
            if(millis()-t0>rate && dist<x && cont>=1)
            {
                state = PRINT;
            }
            break;
        case PRINT:
            Serial.print(val1);
            Serial.print('\t');
            Serial.print(val2);
            Serial.print('\n');
            t0=millis();
            state = ESPERA;
            break;
    }
}

```

```

void toque(int pinBotao)
{
    const int CLICK = 0, SOLTA = 1, PRINT=2;
    static int state = CLICK;
    static bool botao;
    switch (state)
    {
        case CLICK:
            botao = digitalRead(pinBotao);
            if(botao==LOW)
            {
                state = SOLTA;
            }
            break;
        case SOLTA:
            botao = digitalRead(pinBotao);
            if( botao==HIGH)
            {
                state = PRINT;
            }
            break;
        case PRINT:
            Serial.println("S2 premido");
            cont=0;
            state = CLICK;
            break;
    }
}

```

```

void led()
{
    static const int ESPERA=0, OFF=1,ON=2;
    static int state=ESPERA;
    switch(state)
    {
        case ESPERA:
            if(cont>=1)
            {
                state=OFF;
            }
            else
            {
                state=ON;
            }
            break;
        case OFF:
            digitalWrite(LED,LOW);
            state=ESPERA;
            break;
        case ON:
            digitalWrite(LED,HIGH);
            state=ESPERA;
            break;
    }
}

```

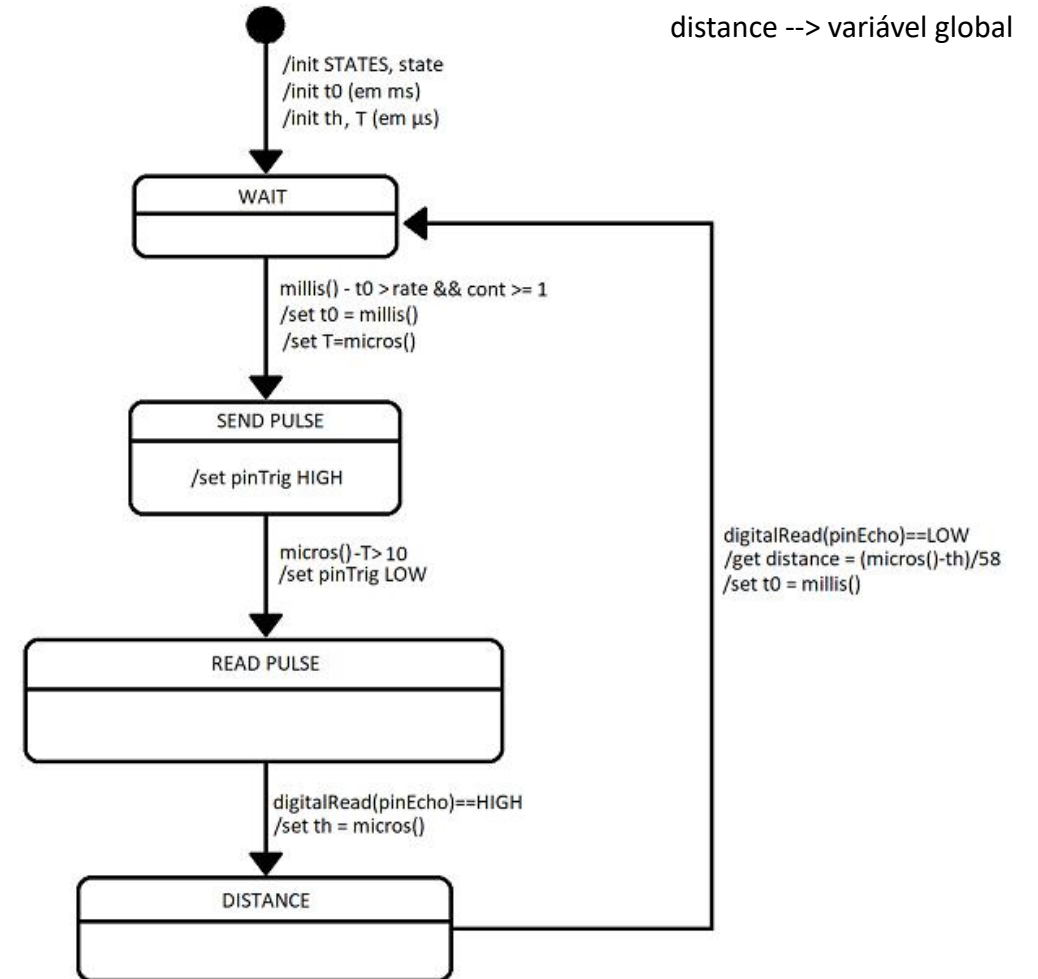
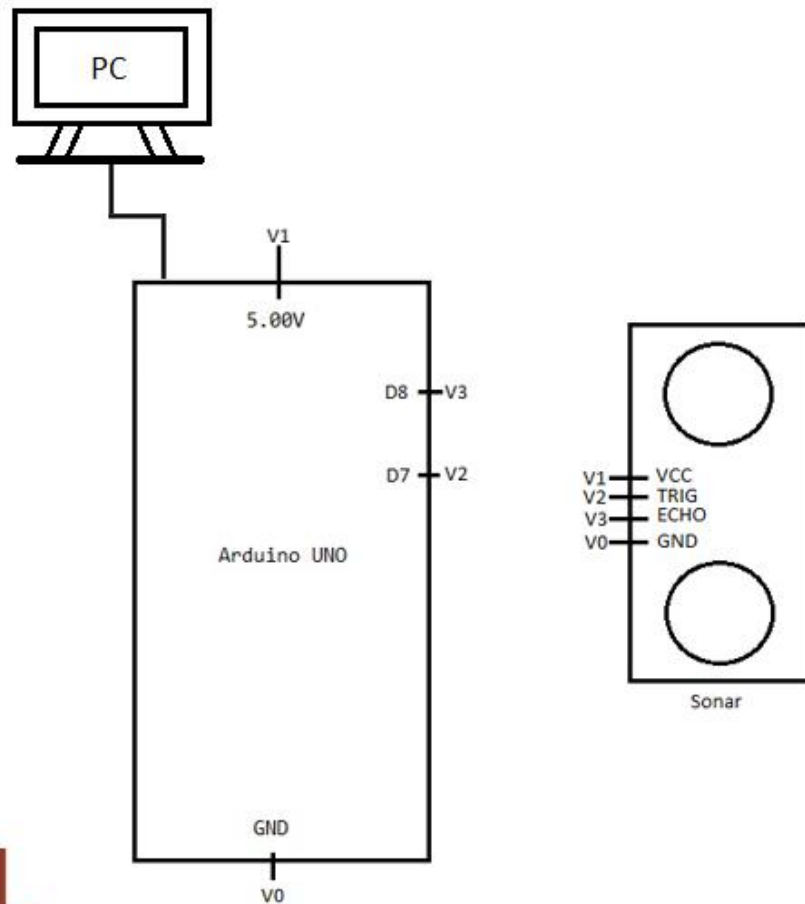
```

void loop()
{
    led();
    contador(S1);
    distance(TRIGG,ECHO, 10);
    melodia(100, n1, n2, n3);
    gerador( 10, 5, tipo_onda);
    letra();
    printCSV(2, dist, cont);
    toque(S2);
}

```

Código Lab 6 - por partes

Medidor de distâncias (sonar)

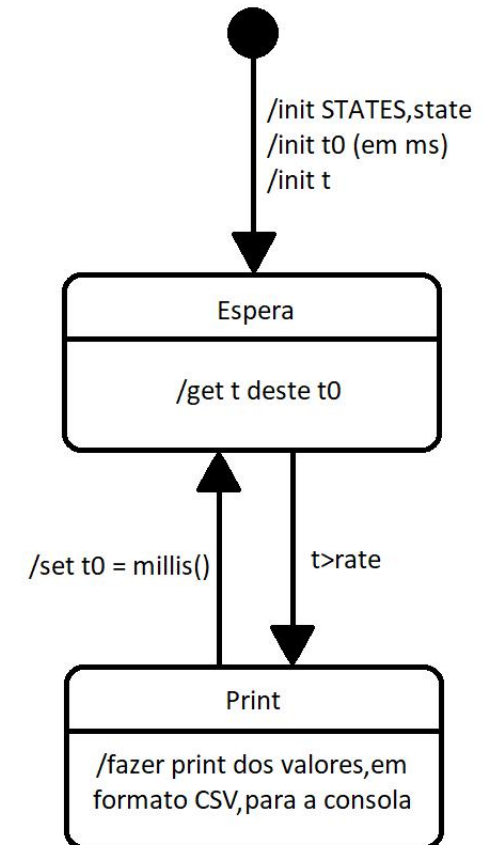
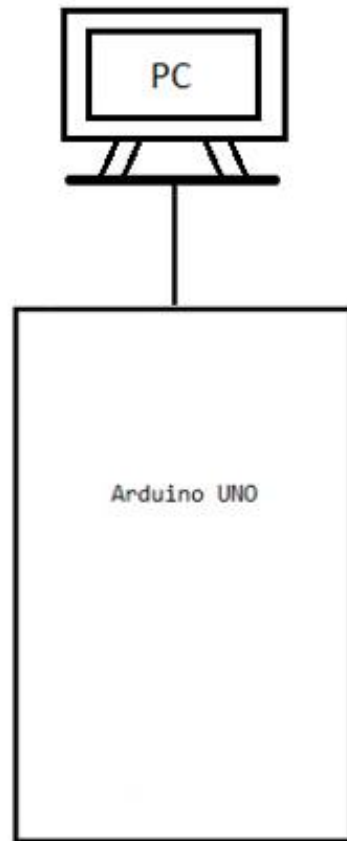


Medidor de distâncias (sonar) - code

```
void distance(int pinTrig, int pinEcho, int rate)
{
    const int WAIT=0, SEND_PULSE = 1 , READ_PULSE = 2 , DISTANCE = 3;
    static int state = SEND_PULSE;
    static unsigned long t0 = millis(), T = micros(), th=micros();
    rate=1000/rate;
    switch (state)
    {
        case WAIT:
            if ( millis()-t0>rate && cont >= 1)
            {
                t0=millis();
                T = micros();
                state = SEND_PULSE;
            }
            break;
        case SEND_PULSE:
            digitalWrite(pinTrig, HIGH);
            if ( micros()-T>10)
            {
                digitalWrite(pinTrig, LOW);
                state = READ_PULSE;
            }
            break;
    }
```

```
        case READ_PULSE:
            if(digitalRead(pinEcho)==HIGH)
            {
                th=micros();
                state = DISTANCE;
            }
            break;
        case DISTANCE:
            if(digitalRead(pinEcho)==LOW)
            {
                dist = (micros()-th)/58.0;
                //Serial.println (dist); //feito apenas para testar o sonar
                t0=millis();
                state = WAIT;
            }
            break;
    }
}
```

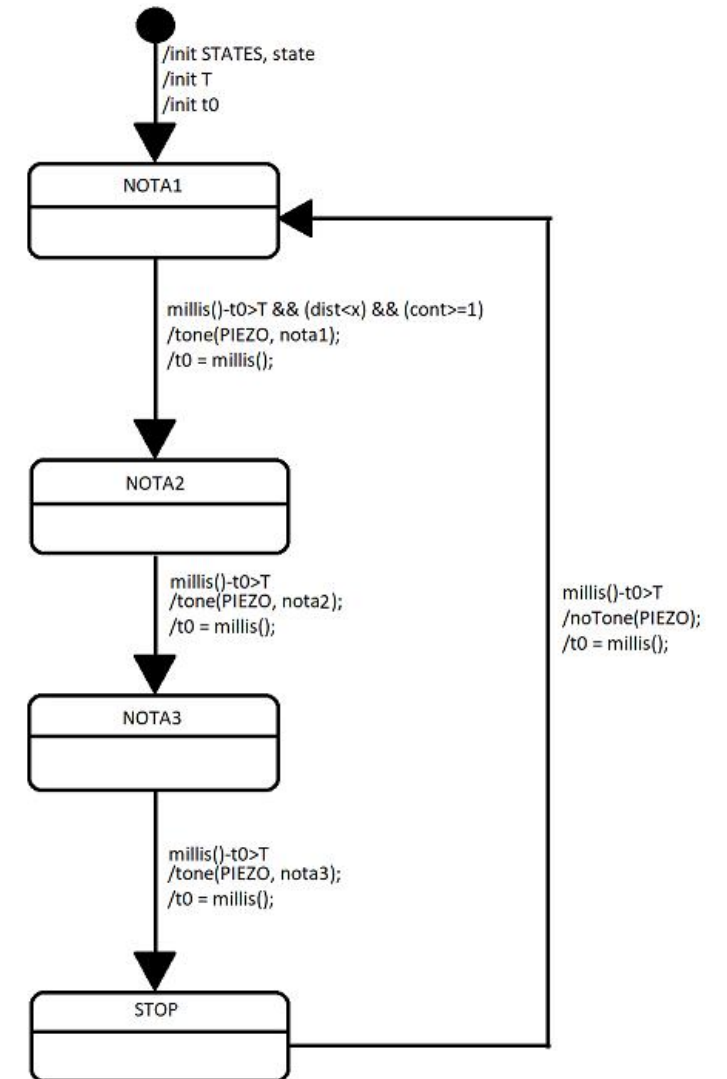
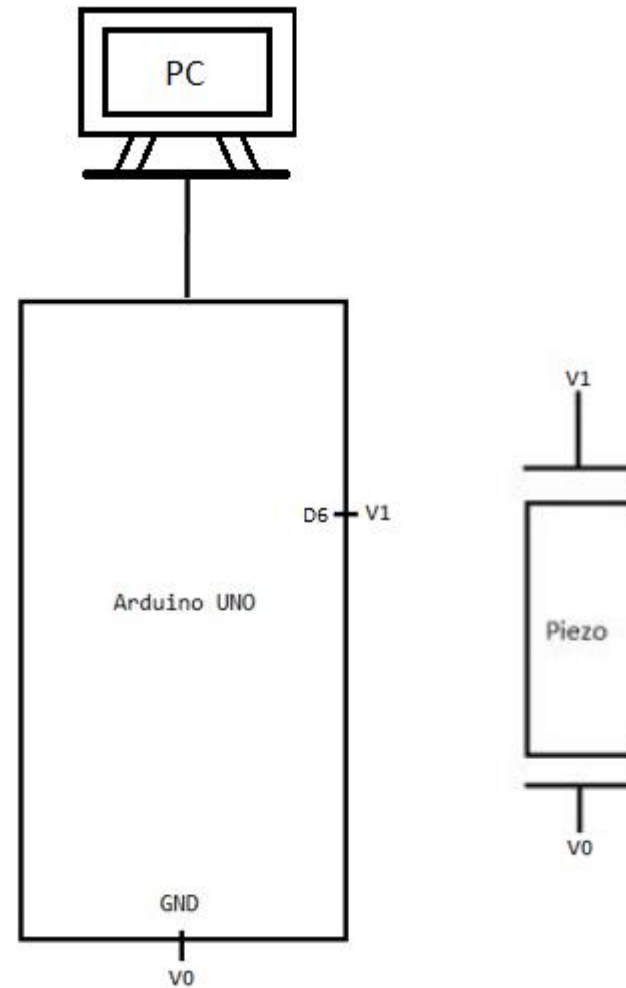
Impressora para consola (formato CSV --> números aleatórios)



Impressora para consola (formato CSV) - code

```
void printCSV(int rate, float val1, int val2)
{
    static const int ESPERA=0, PRINT=1;
    static int state=ESPERA;
    static unsigned long t0= millis();
    rate=1000/rate;
    switch(state)
    {
        case ESPERA:
            if(millis()-t0>rate && dist<x && cont>=1)
            {
                state = PRINT;
            }
            break;
        case PRINT:
            Serial.print(val1);
            Serial.print('\t');
            Serial.print(val2);
            Serial.print('\n');
            t0=millis();
            state = ESPERA;
            break;
    }
}
```

Tocar melodia (piezo)

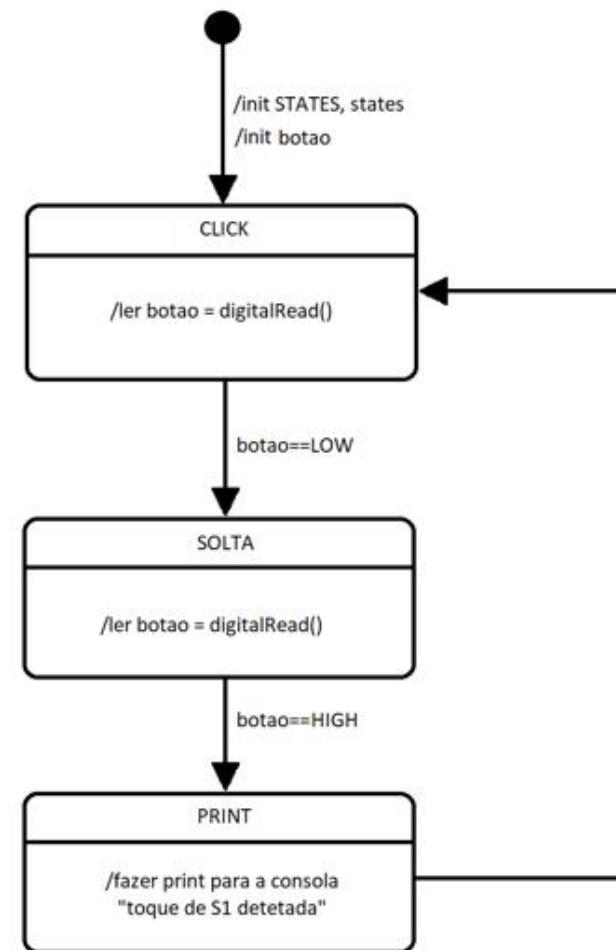
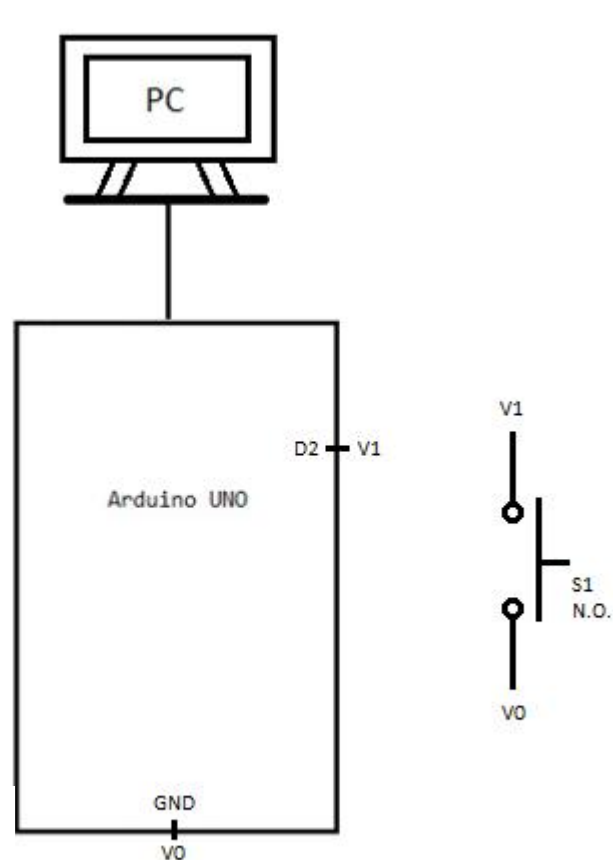


Tocar melodia (piezo) - code

```
void melodia(int duracao, int nota1, int nota2, int nota3){
    unsigned long T=duracao; //Tempo que toca cada nota
    static const int NOTA1=0, NOTA2=1, NOTA3=2, STOP=3;
    static int state=NOTA1;
    static unsigned long t0=micros();
    switch (state)
    {
        case NOTA1:
            if (millis()-t0>T && (dist<x) && (cont>=1)){
                tone(PIEZO, nota1);
                t0 = millis();
                state = NOTA2;
            }
            break;
        case NOTA2:
            if (millis()-t0>T){
                tone(PIEZO, nota2);
                t0 = millis();
                state = NOTA3;
            }
            break;
    }
```

```
        case NOTA3:
            if (millis()-t0>T){
                tone(PIEZO, nota3);
                t0 = millis();
                state = STOP;
            }
        case STOP:
            if (millis()-t0>T){
                noTone(PIEZO);
                t0 = millis();
                state = NOTA1;
            }
            break;
    }
}
```

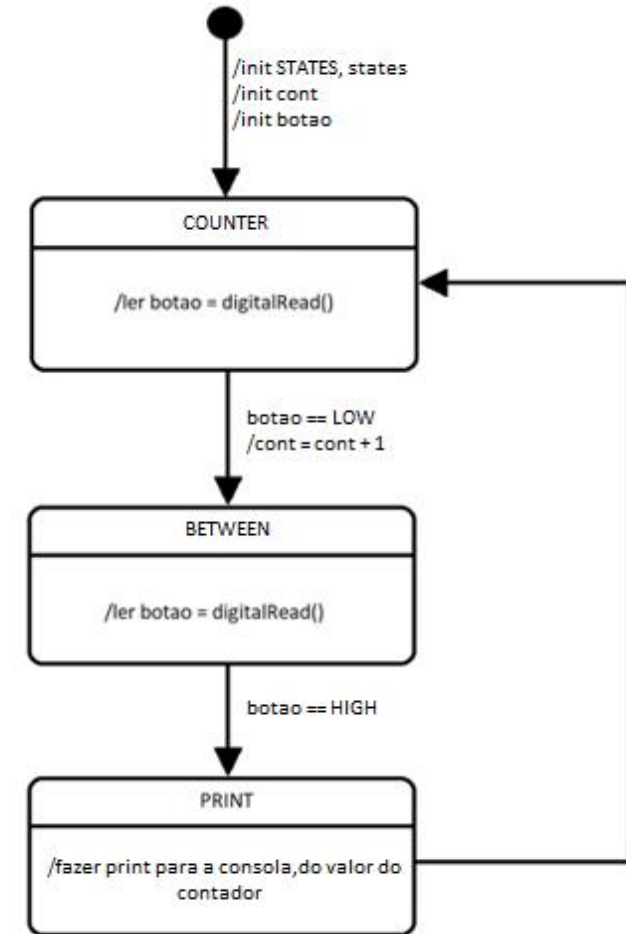
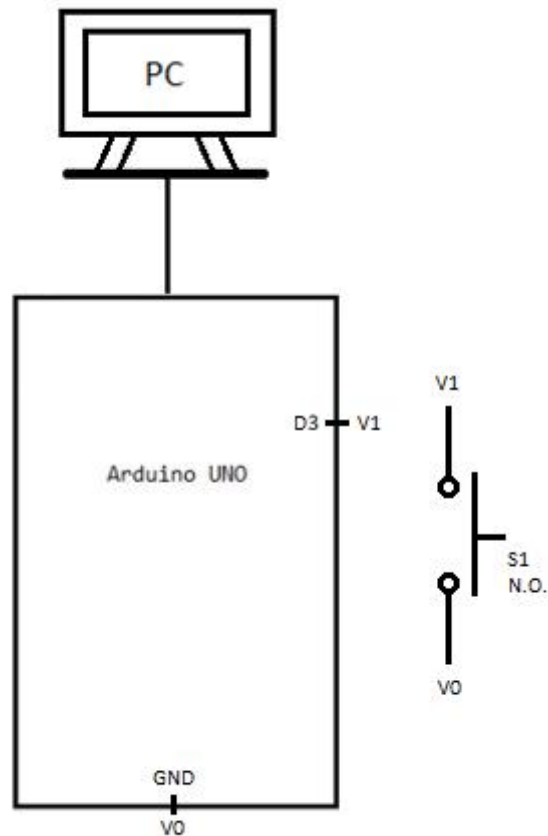

Detetor toque (botão)



Detetor toque (botão) - code

```
void toque(int pinBotao)
{
    const int CLICK = 0, SOLTA = 1, PRINT=2;
    static int state = CLICK;
    static bool botao;
    switch (state)
    {
        case CLICK:
            botao = digitalRead(pinBotao);
            if(botao==LOW)
            {
                state = SOLTA;
            }
            break;
        case SOLTA:
            botao = digitalRead(pinBotao);
            if( botao==HIGH)
            {
                state = PRINT;
            }
            break;
        case PRINT:
            Serial.println("S2 premido");
            cont=0;
            state = CLICK;
            break;
    }
}
```

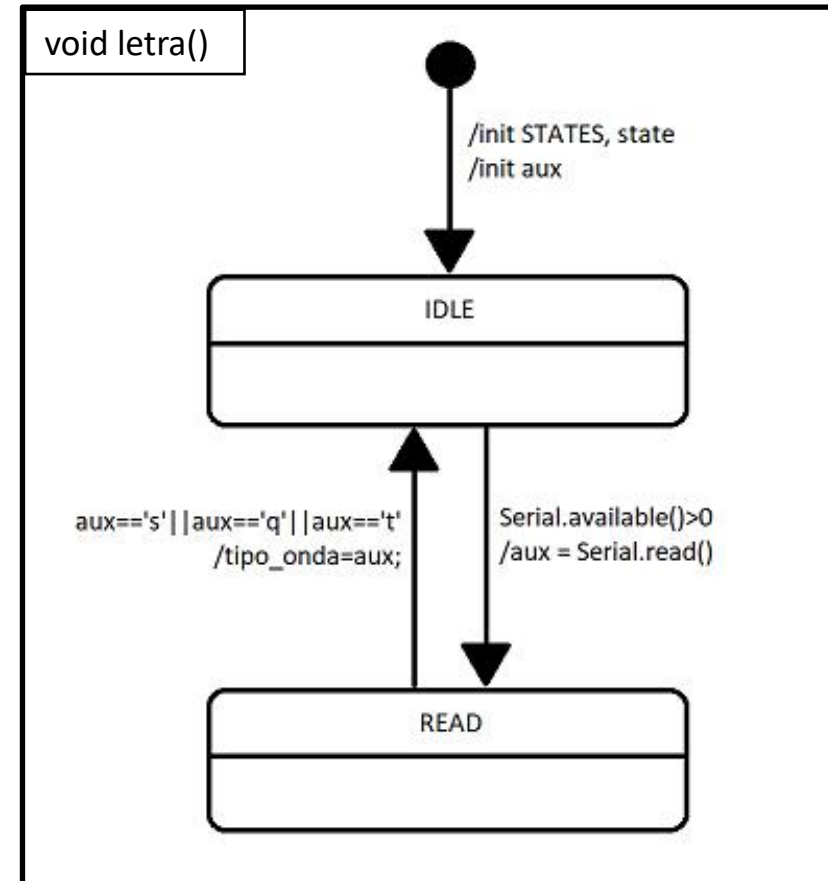
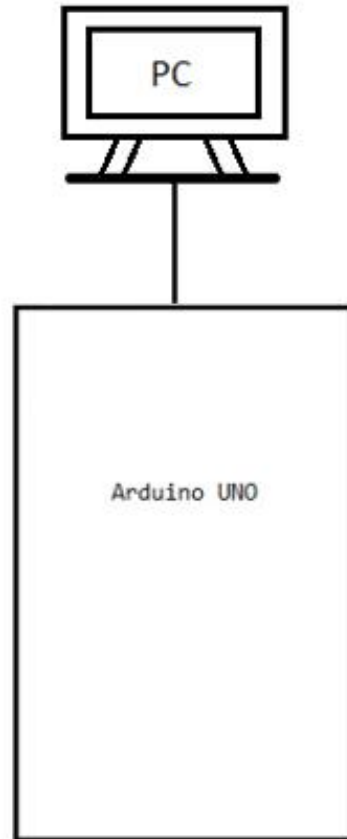
Contador (botão)



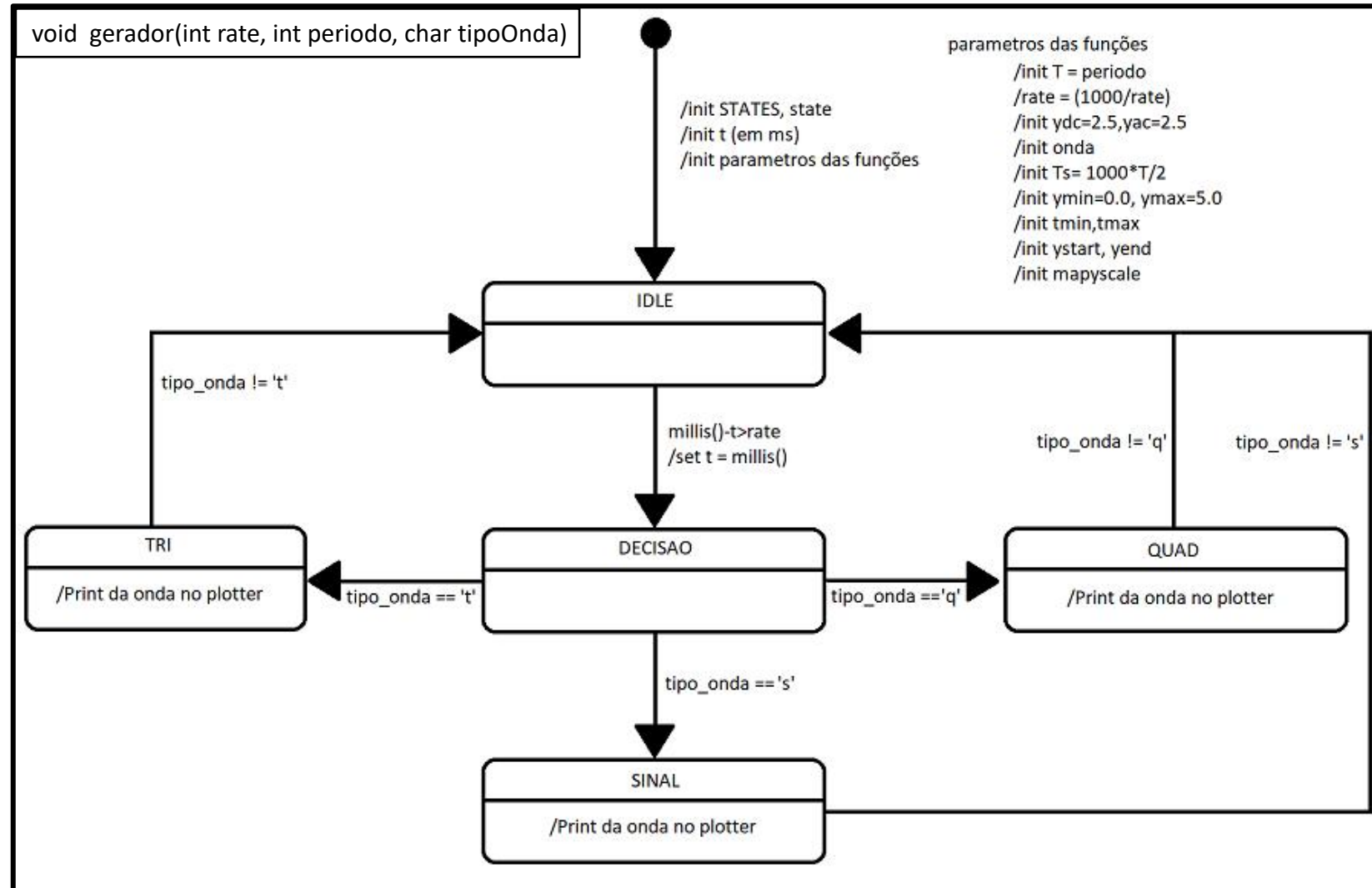
Contador (botão) - code

```
void contador(int pin)
{
    const int COUNTER = 0 , BETWEEN=1;
    static int state = COUNTER;
    static bool botao;
    switch (state)
    {
        case COUNTER:
            botao = digitalRead(pin);
            if(botao==LOW)
            {
                cont= cont + 1;
                state= BETWEEN;
            }
            break;
        case BETWEEN:
            botao = digitalRead(pin);
            if (botao==HIGH)
            {
                state=COUNTER;
            }
            break;
    }
}
```

Gerador sinais (onda sinusoidal triangular ou quadrada)



Gerador sinais (onda sinusoidal triangular ou quadrada)



Gerador sinais (onda sinusoidal, triangular ou quadrada) - code

```
void letra()
{
    static const int IDLE = 0, READ = 1;
    static int state = IDLE;
    static char aux;
    switch (state)
    {
        case IDLE:
            if(Serial.available()>0){
                aux = Serial.read();
                state = READ;
            }
            break;
        case READ:
            if(aux=='s' || aux=='q' || aux=='t')
            {
                tipo_onda=aux;
                state=IDLE;
            }
            break;
    }
}
```

```
void gerador(int rate, int periodo, char tipoOnda){
    static const int IDLE=0, DECISAO=1, SINAL=2, QUAD=3, TRI=4;
    static unsigned long t=millis();
    static int state = IDLE;
    int T=periodo;
    rate = 1000/rate;
    //sinusoidal:
    float ydc=2.5, yac=2.5;//max 5 e min 0
    float onda=0;
    int Ts=1000*T/2; //semiperiod in milisegundos (quadrada e triangular)
    float ymin=0.0,ymax=5.0; //quadrada e triangular
    unsigned long tmin, tmax;

    float ystart,yend;//y value at start/end of semicycle
    float mapyscale=100;

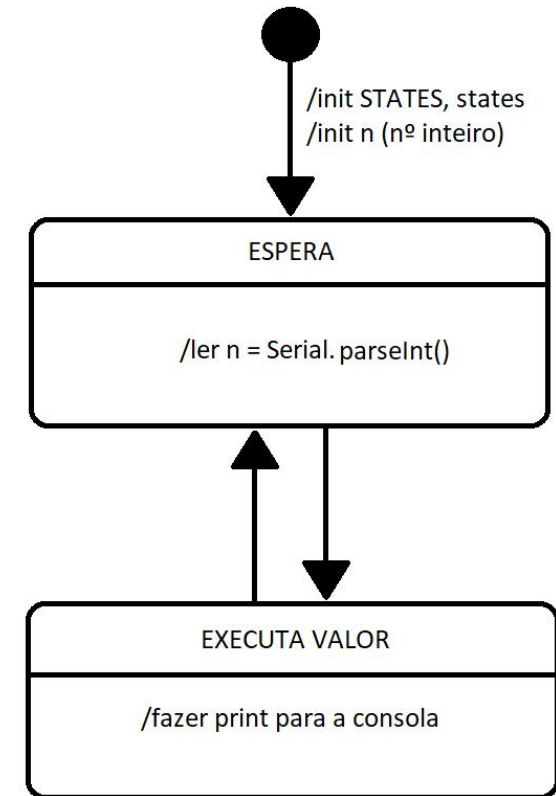
    switch (state)
    {
        case IDLE:
            if(millis()-t>rate && dist>=x)
            {
                t=millis();
                state=DECISAO;
            }
            break;
        case DECISAO:
            if(tipo_onda == 's')
            {
                state=SINAL;
            }
            if(tipo_onda == 'q')
            {
                state=QUAD;
            }
            if(tipo_onda == 't')
            {
                state=TRI;
            }
            break;
    }
```

```
    case SINAL:
        t=millis();
        onda = ydc+yac*sin(6.28*t/1000/T);
        Serial.println(onda);
        if((tipo_onda != 's' )|| (dist>=x))
        {
            state=IDLE;
        }
        break;
    case QUAD: //ymin=0.0,ymax=5.0;; Ts=1000*T/2;
        t=millis();
        if((t/Ts)%2==0) Serial.println(ymin);
        else Serial.println(ymax);
        if((tipo_onda != 'q')||( dist>=x))
        {
            state=IDLE;
        }
        break;
    case TRI:////ymin=0.0,ymax=5.0;; Ts=1000*T/2;
        tmin=t/Ts*Ts; //start time of each semicycle
        tmax=tmin+Ts; //end time of each semicycle
        if((tmin/Ts)%2==0)
        {
            ystart=ymin;
            yend=ymax;
        }else{
            ystart=ymax;
            yend=ymin;
        }
        t=millis();
        onda= map(t,tmin,tmax,mapyscale*ystart,mapyscale*yend)/mapyscale;
        Serial.println(onda);
        if((tipo_onda != 't')|| (dist>=x))
        {
            state=IDLE;
        }
        break;
    }
```


LED()

```
void led()
{
    static const int ESPERA=0, OFF=1,ON=2;
    static int state=ESPERA;
    switch(state)
    {
        case ESPERA:
            if(cont>=1)
            {
                state=OFF;
            }
            else
            {
                state=ON;
            }
            break;
        case OFF:
            digitalWrite(LED,LOW);
            state=ESPERA;
            break;
        case ON:
            digitalWrite(LED,HIGH);
            state=ESPERA;
            break;
    }
}
```

Recetor números inteiros (porta serial)



Recetor números inteiros (porta serial) - code

```
void receiveInt()
{
    static const int ESPERA = 0, EXECUTA_VALOR=1;
    static int state = ESPERA;
    static int n=0;
    switch (state)
    {
        case ESPERA:
            if(Serial.available() > 0)
            {
                n = Serial.parseInt();
                state= EXECUTA_VALOR;
            }
            break;
        case EXECUTA_VALOR:
            Serial.println(n);
            state = ESPERA;
            break;
    }
}

void loop()
{
    receiveInt();
}
```